LINUX INTERVIEW QUESTIONS

1. **How do you check disk space usage in Linux?**

   To check disk space in Linux, I use `df -h` for filesystem usage and `du` to analyze directory-level consumption."

2. **Where are system logs stored?**

   /var/logs

3. **If a server is not starting properly, how do you debug it?**

   I check server reachability, review system and application logs, verify disk space and inodes, check failing services using systemctl, validate configurations and permissions, and finally restart services while monitoring logs to identify the root cause.

4. **Disk full troubleshooting?**

   I identify the full filesystem using `df -h`, check inode usage, find large directories and files using `du` and `find`, clean up logs or temporary files, restart services if deleted files are held, and extend the disk if required.

5. **Can you explain Linux file permissions and how you would give execute permission to a user?**

   Linux file permissions define read, write, and execute access for owner, group, and others. Execute permission can be given using `chmod`, for example `chmod u+x file.sh` or `chmod 755 file.sh`.

   To give execute permission to **owner only** → chmod u+x file.sh
   To give execute permission to **group** → chmod g+x file.sh
   To give execute permission to **others** → chmod o+x file.sh
   To give execute permission to **all users** → chmod +x filename

6. **How do you change file permissions?**

   Using chmod command

7. **What does 777 permission mean?**

   We are giving read, write & execute permissions to all users i.e. Owner(user), Group, Others.

   (OR)

   777 permission means read, write, and execute access for owner, group, and others—full permissions for everyone.

8. **What are the numeric values for read, write, and execute permissions?**

   Read → 4
   Write → 2
   Execute → 1

9. **What are the default permissions of a file and directory?**

   File → 644

   Directory → 755

10. **How do you change the ownership of a file?**

    File ownership in Linux is changed using the `chown` command, for example `chown user:group filename`.

11. **Root user but cannot create file?**

    If root cannot create a file, I check whether the filesystem is read-only, disk or inode usage is full, SELinux is blocking access, or the filesystem is corrupted or mounted from NFS.

12. **How do you create a user and assign it to a group?**

    To create a user and assign it to a group, I use `useradd -g group username`, set a password with `passwd`, and use `usermod -aG group username` for additional groups.

13. **How do you create a user in Linux?**

    To create a user, we use useradd or adduser command

14. **Where are user details stored in Linux?**

    User details in Linux are stored in `/etc/passwd`, password information in `/etc/shadow`, and group details in `/etc/group` and `/etc/gshadow`.

15. **How do you troubleshoot disk space issues step by step?**

    I first identify the full filesystem using `df -h`, then locate the top space-consuming directories using `du`. I drill down to large files, check logs under `/var/log`, verify inode usage with `df -i`, and look for deleted but open files using `lsof`. After safe cleanup or service restart, I ensure preventive measures like log rotation and monitoring are in place.

16. **Disk free but cannot perform actions?**

    If disk is free but I cannot perform actions, I check inode usage with `df -i`, directory permissions, whether the filesystem is mounted read-only, deleted files still in use using `lsof`, quotas, and SELinux restrictions. These are the most common causes."

17. **How do you find the largest files in a directory?**

    To find the largest files, I use `du -ah` combined with `sort -hr` and `head`. For example: `du -ah /path | sort -hr | head -10`."

18. **How do you check CPU, memory, and running processes in Linux?**

    I use `top` or `htop` to monitor real-time CPU, memory, and processes, `free -h` for memory usage, `ps -ef` to list running processes, and `uptime` to check load average.

19. **How do you check memory usage?**

    I use `free -h` to check overall memory usage, `top` or `htop` to monitor real-time memory consumption by processes, and `ps` commands to identify memory-intensive processes.

20. **How do you check CPU usage?**

    I use `top` or `htop` to monitor real-time CPU usage, `uptime` for load average, `sar -u` for historical data, and `ps` commands to identify top CPU-consuming processes.

21. **What is free command?**

    The `free` command is used to check system memory usage in Linux. I usually run `free -h` to view total, used, free, available, and swap memory in a human-readable format.

22. **top vs htop?**

    `top` is a default Linux command for real-time process monitoring, while `htop` is an enhanced, user-friendly version with better visualization, per-core CPU usage, and easier process management.

23. **Troubleshoot slow server?**

    I check CPU, memory, disk, and network usage using `top`, `free`, `df`, and `uptime`, identify the resource-consuming process, review logs, and take corrective action based on the root cause.

24. **Memory issue commands?**

    To troubleshoot memory issues, I use `free -h` to check overall usage, `top` or `htop` to identify memory-heavy processes, `ps aux --sort=-%mem`, `swapon -s` for swap usage, and `dmesg` to check OOM events.

25. **What is zombie process?**

    A zombie process is a completed process whose exit status has not been collected by its parent, leaving a defunct entry in the process table.

26. **Where are system logs stored?**

    System logs in Linux are primarily stored under `/var/log`. On system-based systems, logs can also be accessed using `journalctl`.

27. **What kind of logs are in /var/log/messages?**

    `/var/log/messages` contains general system logs such as service messages, kernel warnings, hardware events, network messages, and cron activity, excluding authentication logs

28. **How do you check live logs on a server?**

    I check live logs using `tail -f /var/log/messages` for log files or `journalctl -f` on systemd-based systems to monitor logs in real time.

29. You are unable to SSH into a server. How will you troubleshoot?

I check network connectivity and port access, verify SSH service and firewall rules, validate SSH configuration and user credentials, review SSH logs, and use verbose SSH to identify the exact failure point.

30. How do you check network connectivity between two servers?

I check connectivity using `ping` for reachability, `nc` or `telnet` to test specific ports, `traceroute` for routing issues, and verify DNS and firewall rules on both servers.

31. How to see listening ports?

I use `ss -tulnp` to view all listening TCP and UDP ports along with the associated processes. On older systems, I use `netstat -tulnp`.

32. Networking issue commands?

For networking issues, I use `ping` and `traceroute` for connectivity, `nc` for port checks, `nslookup/dig` for DNS, `ss` or `netstat` for connections, and `tcpdump` for packet-level analysis.

33. The server is not coming up after reboot. What steps will you take?

I check server power and console access, review boot and kernel errors, verify disk and filesystem health, check failed services and `/etc/fstab`, analyze logs, and roll back recent changes if required.

34. If a server is not starting properly, how do you debug it?

I first determine whether it's an OS boot issue or a service issue, then I check reachability/console, review boot logs with `journalctl -xb` and `dmesg`, identify failed services with `systemctl --failed`, validate disk/inodes, ports, permissions, and dependencies, and restart the service while tailing logs to pinpoint the root cause.

Cheat sheet:

```
journalctl -xb
systemctl --failed
systemctl status <svc>
journalctl -u <svc> -n 200
dmesg -T | tail
df -h; df -i
ss -tulnp
lsof | grep deleted
```

35. A crontab job is failing. How would you debug it?

I check cron logs, validate the crontab entry and user, run the script manually, ensure executable permissions, use absolute paths, redirect output, and verify environment variables.

36. **Write a shell script to print the largest of three numbers.**

```
#!/bin/bash

a=$1
b=$2
c=$3

if [ "$a" -ge "$b" ] && [ "$a" -ge "$c" ]; then
  echo "Largest number is: $a"
elif [ "$b" -ge "$a" ] && [ "$b" -ge "$c" ]; then
  echo "Largest number is: $b"
else
  echo "Largest number is: $c"
fi
```

37. **What is the purpose of #!/bin/bash in a script?**

`#!/bin/bash` specifies the Bash interpreter for the script, ensuring it runs with Bash regardless of the user's default shell.

38. **How do you pass arguments to a shell script?**

Arguments are passed from the command line and accessed inside the script using positional parameters like `$1`, `$2`, `$#`, and `$@`.

39. **How shell script executed?**

A shell script is executed either by making it executable and running `./script.sh`, or by invoking it with a shell like `bash script.sh`, where the interpreter executes the script line by line.

40. **What is -e in shell?**

`set -e` makes a shell script exit immediately when any command fails, helping prevent further execution after an error.

41. **Write a shell script you have used in your project.**

In projects, I used shell scripts for housekeeping and monitoring — for example, a cron-based script that checks disk usage using `df`, cleans old logs using `find -mtime`, and generates a report. If usage crosses a threshold, it logs top consumers using `du | sort` and alerts the team.

42. **How do you create a file in Linux?**

We can create a file in Linux using `touch`,

43. **How do you create a directory?**

We create a directory in Linux using the `mkdir` command, and with the `-p` option we can also create parent directories automatically.

44. **What is the difference between soft link and hard link? When would you use each?**

    A hard link points directly to the file's inode and doesn't break if the original is deleted, while a soft link points to the file path and breaks if the target is removed; hard links are used for data integrity, soft links for flexibility and shortcuts.

45. **What happens when you run hostnamectl?**

    `hostnamectl` displays and manages system hostname information such as static and transient hostname, OS, kernel, and architecture on systemd-based Linux systems

46. **Top Linux commands used?**

    The most used Linux commands include `ls`, `cd`, `cp`, `mv`, `rm`, `grep`, `chmod`, `chown`, `ps`, `top`, `df`, `free`, `ping`, `ss`, `systemctl`, and `journalctl` for file, process, disk, network, and service management.